PART VII

Tools

Optional but powerful: the Emulator for replaying and analyzing your loop's decisions.

CHAPTER 11

The Emulator on Your PC

11.1 What the Emulator does

The Emulator is a Python tool that replays your AAPS logfiles on your PC. For any time window you pick, it can:

- Show you every loop decision — what autoISF saw, what components contributed, what SMB was requested, whether anything was capped.
- Ask "what-if" questions — if I had used `bgAccel_ISF_weight = 0.045` instead of `0.035` last Tuesday, how would each meal's first SMB have been different?
- Produce outputs in multiple formats: a full-detail text log, a dense CSV table, a visual PDF chart, and a "delta" analysis for acceleration-driven decisions.

The key insight: it uses your real, already-recorded CGM and insulin data. Nothing hypothetical. Just "given what actually happened, what would the loop have decided under different settings?"

11.2 What it's good for — and what it's not

Good uses:

- Finding a better `bgAccel_ISF_weight` by comparing several candidates against the same meal.

- Seeing which autoISF component (acce / pp / dura / bg) was really driving the SMB size at a given moment.
- Confirming that a tuning change would help without committing to it.
- Analyzing a meal retroactively — *why* did that plateau last so long?

Not good for:

- Predicting the downstream bg curve after a changed setting. The Emulator can only replay; it cannot simulate how a bigger SMB at 12:30 would have changed the 13:00 bg reading.
- Replacing iteration across multiple meals. One meal "optimized" in the Emulator is still one meal — validate across 2–3 different meal types before committing.
- Tuning from first principles. The Emulator helps you refine hypotheses; it does not generate them.

11.3 Installation — high-level

> ⚠️
>
> FOLLOW THE DEVELOPER'S MOST CURRENT INSTRUCTIONS.
>
> File names, branches, and Python versions change. The authoritative source is always:
>
> https://github.com/ga-zelle/APS-what-if
>
> Look at the latest "Instructions determine_basal emulator" PDF and the "How to run the emulator on the phone" PDF under Documentation in English.

The installation involves four broad steps:

1. SET UP THE FOLDER STRUCTURE

Suggested layout (names are arbitrary):

```
Logfiles_Emulator/
├── AAPS_logs/          ← copy your phone's AAPS logfiles here
├── Emulator/           ← downloaded Python files live here
└── Emulator_Studies/
    ├── Study_1/        ← one emulation project = one folder
    ├── Study_2/
    └── ...
```

> 💡 *Copy AAPS logfiles from your phone regularly — they auto-delete after roughly 2 weeks (much faster on 1-minute CGM setups). Keep a month-by-month archive, and alongside each month put a Nightscout Reporter PDF — it makes finding "interesting days" for analysis much easier.*

2. DOWNLOAD THE PYTHON FILES

Two sources, both on GitHub under `ga-zelle`:

- **`APS-what-if/software`** — the emulator itself (a `.config` and four `.py` files).
- **`Scan-APS-logfiles`** — two more `.py` files (scan helpers).

Match the branch version to your AAPS + autoISF version. If you're on AAPS 3.3.3a + autoISF 3.1.0, use the `A3.3.3a+aisf3.1.0` branch. If you can't get the emulator to run, check for a newer file (even with the same name — updates may fix Android-OS-specific issues).

For 1-minute CGM users (Libre 3 via Juggluco), use `1minute_emulator_std.config`. For standard 5-minute CGMs, use `5minute_emulator_std.config`.

3. CREATE A DESKTOP LAUNCHER

Make a shortcut to `emulator_GUI.py` in your Emulator folder, drag it to your desktop, and name it something memorable ("Emulator_start").

4. INSTALL SUPPORTING SOFTWARE

- Notepad++ on your PC (for editing `.vdf` files — see §11.4).
- Python — usually already present. If not, see the developer's install guide.

11.4 Analyzing a logfile — the "no-change" workflow

This is the simplest mode: replay what happened, with no modifications.

CREATE `NOCHANGE.VDF`

A `.vdf` file tells the Emulator what to change for a "what-if" run. An empty `.vdf` is a `noChange.vdf` — replay as-is.

1. Open Notepad++ and save an empty file as `noChange.vdf`.
2. Keep a copy at the top of your `Emulator_Studies/` folder. For each new study, copy it into the Study_n folder.

> ⚠️ Every study folder must contain a **`noChange.vdf`**. Even for what-if runs, the emulator needs the no-change baseline to compare against.

PREPARE YOUR RUN

Open the Emulator (from your desktop launcher). A dialog box appears with three paths to fill in:

1. Project folder — where results go. Usually `Emulator_Studies/Study_n/`.
2. VDF file — for a no-change run, the `noChange.vdf` in that same Study_n folder.
3. Logfiles — browse to any logfile from the day you want to analyze. Then:
    - Replace the time in the filename with `*` (asterisk) → this grabs all logfiles from that day.
    - Click Show matches to verify.
    - Use the bottom time fields to narrow the window. Times are in UTC; convert from your local time (e.g., for Central EU Summer Time, subtract 2 hours).

RUN THE EMULATION

Click Run emulation. If there's a syntax or version error, check the `.log` file first. The Discord channel (https://discord.gg/n3tD5eXExC) is the fastest place to get help.

THE FIVE RESULT FORMATS

The emulation produces multiple output files. Each gives you a different view.

Output	*What it contains*	*Best for*
`noChange.txt`	Full AUTO ISF tab content for every loop decision	Deep-dive investigation — search for "autoISF," "SMB disabled," etc.
`noChange.csv`	Tabular data for every decision, every 5 minutes	Looking at trends across hours; copy into Excel
Spreadsheet extract (manual)	Your annotated and color-coded version of the CSV	Writing up a case study or comparing multiple runs
`noChange.pdf`	Visual chart: bg, iob, iobTH corridor, insulin activity, ISF scatter	Quick eyeball: "did anything weird happen?"
`delta` sheet (newer emulator core)	Short and long average delta analysis	FCL using Automations (Chapter 8, §8.1) based on delta

For most users, the PDF chart + the CSV (in Excel) is the right combination.

📖

TIPS FOR THE CSV WORKFLOW IN EXCEL

- *Add a local-time column next to UTC (e.g., `=UTC_cell + 2/24` for Central European Summer Time).*
- *Convert time format from hh:mm:ss to hh:mm.*
- *Hide columns you don't care about for this analysis; your formulas will still work.*
- *Color-code the rows where: an SMB was given, a particular ISF component dominated, iobTH was exceeded, a TT became active.*

11.5 "What-if" analysis — the `yourChange.vdf` workflow

This is why you installed the Emulator. Ask: "if I had used these settings instead, how would the loop have decided?"

WRITING A `YOURCHANGE.VDF`

The file lists parameter changes. Simplest form — one line per parameter, tab-separated:

```
profile   bgAccel_ISF_weight   profile['bgAccel_ISF_weight']*1.2   ### 20%
stronger
```

That line says: "for this emulation run, pretend `bgAccel_ISF_weight` was 20% bigger than it actually was."

- Factor-based syntax (`* 1.2`, `* 0.8`) is the most common form.
- Multiple lines for multiple parameter changes.
- Save the file with a descriptive name like `1.2_bgAccel_2.0_bgBrake_1.2_dura.vdf`.

More complex VDF syntax (e.g., defining circadian `STAIR_ISF` tables for whole-profile experiments) is in the developer's "Guide to VDF Files for the AAPS Emulator" PDF.

RUNNING IT

Same process as the no-change run (§11.4), but point the middle "VDF file" field at your `yourChange.vdf` instead of `noChange.vdf`. The Emulator runs both — you get both sets of results side-by-side in the same Study_n folder.

READING THE RESULTS

Each output type now comes in two versions (noChange and yourChange):

- **`yourChange.txt`** — for every loop decision, exactly how the changed setting would have altered it.
- **`yourChange.csv`** — the tabular version. Compare against `noChange.csv` side-by-side in Excel (either manually, or using the spreadsheet-based tool `autoISF_factors_performa.ods` in the developer's repo).
- **`yourChange.pdf`** — the visual chart, including markers for every decision where the yourChange value differed from noChange.

The visual chart shows "dark green scatter points" for autoISF ISF under `yourChange` and "light green" for `noChange`. Where they diverge is where your setting change mattered.

WHAT TO LOOK FOR

- The first place your change made a meaningful difference is the most informative. Everything after that would have put the bg curve on a different trajectory, so later comparisons become increasingly hypothetical.
- Which meal phases did the change affect — acceleration? plateau? deceleration?
- Did the change run into any safety limit you'd need to widen?

11.6 Limitations — important to internalize

> ⚠️
>
> EACH EMULATOR RESULT IS A SINGLE-DECISION CALCULATION, NOT A SIMULATION.
>
> If yourChange would have delivered a bigger SMB at 12:30, the Emulator correctly shows that. What it cannot show is: how the extra insulin would have lowered the 12:35 bg reading, and how that would have changed every subsequent decision.
>
> The first meaningful divergence between noChange and yourChange is the most reliable insight. Later divergences are anyone's guess.

This is why:

- Emulator analysis should focus on early, acceleration-phase decisions — where the "first big difference" is most cleanly visible.
- dura_ISF tuning can't be well-emulated — because it depends on plateaus that would themselves be different under different earlier settings.
- You still need real-world validation — the Emulator narrows your hypotheses; the next week of actual meals confirms them.

11.7 Where to get help

- Developer docs: https://github.com/ga-zelle/APS-what-if (look for PDFs under `Documentation in English`).

- Discord: https://discord.gg/n3tD5eXExC (channel: `emulate-aaps`). Fastest place to resolve version or path issues.
- Case studies: the book's case studies 4.1 (pizza tuning), 6.2 (biking day), and 8.2 (futility of tuning from one meal) all use the Emulator and show worked examples.

* * *

CHAPTER 12

The Emulator on Your Phone

The phone Emulator is a lighter-weight companion to the PC Emulator. It's useful for two things:

1. Quickly seeing recent loop decisions in tabular form — a better view than scrolling the AUTO ISF tab.
2. Real-time "what-if" announcements — spoken aloud by the phone, when a changed setting would have produced a different SMB.

The second one is genuinely unique. It lets you test a setting idea "in real life" without committing to it.

12.1 Why the phone Emulator exists

The PC Emulator is great for retrospective analysis. But for real-time tuning — "am I willing to follow this different setting's suggestion, and add this extra SMB manually?" — you need the tool running on the phone that's handling your loop.

The phone Emulator is:

- More limited in output (no PDF charts, simpler CSV).
- Better at real-time feedback.
- The only way to get spoken suggestions (§12.5).

On Android, it runs via QPython 3L. On iPhone (Trio/iAPS), a lighter built-in feature gives you just the tabular view — no full emulator, no speech. See §12.7.

12.2 Installation — high-level

> ⚠️
>
> THE DEVELOPER'S INSTALLATION GUIDE IS THE AUTHORITATIVE SOURCE.
>
> Versions, permissions, and Android OS quirks change. Start there:
>
> https://github.com/ga-zelle/APS-what-if → Documentation in English → Installation Guide PDF

The flow:

1. INSTALL QPYTHON 3L

- Get it from Google Play Store.
- Put the app icon next to your other looping apps.
- Long-press the app icon → App info → exclude it from battery optimization, aggressive app-killing, etc. (Same settings you made for AAPS, NSClient, xDrip, and anything else critical.)

> ⚠️ If QPython gets killed by the OS, the emulator and speech synthesis stop silently. Lock it down like every other critical looping app.

2. COPY THE .PY FILES

Connect the phone via USB, then copy all the emulator `.py` files from your PC's `Emulator/` folder into the phone's `QPython/scripts3/` folder.

Skip `emulator_GUI.py` — that's for the PC desktop launcher only.

3. COPY THE .CONFIG FILE

Copy `5minute_emulator_std.config` (or `1minute_emulator_std.config` for Libre 3) from your PC to the phone at:

```
Internal memory / AAPS / logs / info.nightscout.androidaps /
```

Not the QPython folder — the AAPS logs folder.

4. COPY NOCHANGE.VDF

Same location. The phone Emulator reads `.vdf` files from here.

5. OPTIONAL — MORE .CONFIG VARIANTS

You can have multiple config files — e.g., one that announces everything, one that only announces insulin (no carb warnings). More on this in §12.6.

12.3 The result table on the phone

Once running, QPython shows a compact table of recent loop decisions. This table is the main passive output. You can consult it at any time to see, for the last N decisions:

- Which autoISF category (acce / pp / bg / dura) contributed how much.
- What the effective ISF was.
- What SMB was requested, delivered, or blocked.

The table is much easier to read than scrolling multiple AUTO ISF tab snapshots, especially when tracking a meal in progress.

CUSTOMIZATION

The output table format is defined in the `.config` file. You can customize which columns appear, reorder them, change decimal separators, etc. See the developer's "How to run the emulator on the phone" PDF for the exact syntax.

12.4 What-if analysis on the phone

Same concept as on the PC (§11.5), but running against the current live logfile:

1. Write a `yourChange.vdf` on the PC (§11.5).
2. Copy it to the phone's AAPS logs folder.

3. Select it in the running QPython emulator (via a menu action — see the dev docs).
4. The emulator now runs your change in parallel with the real loop, for each new CGM value.

The results show up in the table and — if you enable it — in spoken announcements.

12.5 Real-time speech synthesis — the killer feature

> ⚠️ iPhone users: this feature is not available. See §12.7 for what Trio and iAPS offer instead.

When the what-if emulator is running and detects that your experimental setting would produce a meaningfully different SMB than the real loop's decision, the phone can speak an announcement:

- *"Extra SMB of 0.8 units recommended."*
- *"Reduce by 0.5 units."*
- *"Carbs required" warnings (useful even without autoISF).*

You can then:

1. Look at your glucose curve and the developing situation.
2. Decide whether to follow the suggestion by manually adding a bolus of the suggested size. (You'll need to temporarily re-enable the insulin button in AAPS Preferences → Overview → Buttons → Insulin = ON during test phases.)
3. Watch what happens over the next hour.

After a few days of this, you develop intuition for whether the investigated change really deserves to be folded into your active settings.

> ⚠️ Your settings must work across a variety of meals. Don't be seduced by one meal where the what-if suggestion was clearly better — see Case Study 8.2 on the futility of tuning from one extreme meal.

12.6 Managing announcements

TIME WINDOWS FOR SPEECH

The `.config` file defines when announcements are allowed. Default: 07:00–23:00 local time (not UTC). Edit `5minute_emulator_std.config` on your PC with Notepad++, then copy the updated file back to the phone's AAPS logs folder.

The config has three independent time windows:

- Line 1: when "carbs required" announcements are allowed.
- Line 2: when "extra bolus need" announcements are allowed.
- Line 3: when "reduce bolus" announcements are allowed.

Set each to match how useful that type of alert is for you.

SILENCING THE EMULATOR TEMPORARILY

Options, in order of preference:

1. Switch to `noChange.vdf` as the active VDF. The emulation continues (you still get the result table), but no what-if comparison runs → no speech.
2. Use a "silent" custom `.config` with very narrow time windows for announcements, and switch to it via the emulator's menu.
3. Silence the phone (mute + do not disturb) — blunt, shuts off other alerts too.
4. Kill QPython (force-stop the app) — emulation stops entirely. No tabular data for that period.

Option 1 is the best default. It keeps you collecting data while silencing the proactive announcements.

12.7 iPhone users: Trio and iAPS

iOS-based autoISF variants cannot run the full emulator (no QPython equivalent, no Python-based PC tool-chain on iOS). Two lighter options are available instead:

BUILT-IN TABLE VIEW

Both Trio's and iAPS's autoISF ports integrate a compact table showing the recent loop decisions and the contribution of each autoISF category (acce / pp / bg / dura). Not as powerful as the full emulator, but enough for quick sanity-checks.

Access: in Trio, double-click "Statistics." In iAPS, similar — check the current readme for your version.

NO SPEECH SYNTHESIS

The real-time "what-if" with spoken announcements is not available on iPhone. Workarounds:

- Do PC-side analysis with the emulator if you have an Android device somewhere (not the looping phone — any Android for analysis work).
- Use the phone's on-screen table view to watch recent decisions during a meal.
- Consult the autoISF Quick Guide and community for guidance.

Future updates may close this gap. Check https://github.com/mountrcg/Trio and https://github.com/mountrcg/iAPS for current status.

12.8 Summary

- The phone Emulator extends the PC Emulator with a real-time interface.
- Its best feature is spoken what-if suggestions during active meals.
- Installation is more fiddly than the PC emulator (QPython quirks, file placement). Budget an evening.
- On iPhone, you get a compact tabular view but no full emulator or speech.
- The biggest pitfall is letting Android's battery optimizer kill QPython silently.

* * *

BACK MATTER

Final Remark & Appendices

Closing thoughts, a glossary of terms, and an index of case studies.

Final Remark

This is an exciting time to be part of the open-source T1D community. The tools described in this book — autoISF, the Emulator, the Automations ecosystem — exist because users built them for each other and shared the work freely. If you build something, contribute it back. If you learn something, share it.

Carefully weigh for yourself what your entry point will be. And then — once you've done the work of building a system that handles your meals hands-off — *just eat happily ever after.*

— *The authors*

Appendix A — Glossary

Key terms used in this book, organized alphabetically. Descriptions are concise; for deeper reading, follow the linked references in the chapters where the terms first appear.

A

AAPS (AndroidAPS) — Open-source Artificial Pancreas System for Android phones. Bluetooth-connected with a pump and a CGM. Supports the broadest choice of pumps and CGMs of any DIY looping option.

AAPS Client — Companion app for remote monitoring and limited control of an AAPS phone (e.g., parent → child). iOS equivalent: Loop Caregiver.

acce_ISF → see `bgAccel_ISF`.

acceleration — Mathematical analysis of the bg curve that detects the earliest signs of a rise. autoISF uses parabola-fitted acceleration; simpler systems use growing bg deltas (which lag by 10–20 minutes).

Activity Monitor — autoISF feature that uses phone step count to auto-adjust sensitivity over a ~60-minute window. Capped at +20% (inactivity) / & 30% (activity). See §6.6.

aggressiveness (of the loop) — How strongly the loop responds to glucose deviations. Modulated by ISF, bg target, %profile, iobTH, and exercise mode settings.

AIMI — AAPS-derived dev variant with a Meal Announcement workflow. Also ported into Trio. See §8.3.

algorithm — The set of calculations and safety checks a loop performs every 5 minutes (or every minute for 1-minute CGMs) to decide insulin delivery. The oref family (AAPS, Trio, iAPS) and iOS Loop use different algorithms.

AMA (Advanced Meal Assist) — Older oref approach to handling meal carbs via increased temp basal, without SMBs. Superseded by SMB+UAM in modern oref.

Android Studio — Free developer software needed to build and maintain your personal AAPS installation.

Anubis — DIY re-engineered transmitter for the Dexcom G6 CGM. Unlimited lifespan (battery change every ~6 months); won't time out at 10 days.

apk — Android application package. The compiled file used to install AAPS on your phone.

APS — Artificial Pancreas System. Umbrella term for insulin delivery systems that regulate bg to a target based on CGM data.

autoCR — Automated carb-ratio adjustment. Pilot feature in iAPS; not typically useful in advanced oref FCL.

autoISF — The version of oref SMB+UAM used in this book. Adapts ISF sharply to the developing glucose curve (acceleration, delta, level, stuck-at-high). Only available in dev variants of AAPS, Trio, and iAPS.

Auto Bolus / Automatic Bolus — In iOS Loop terminology, a small automatically-delivered bolus for faster bg correction than TBR alone. Oref equivalent: SMB.

Automation — AAPS feature that lets you define patterns in your data (conditions) and actions to take (loop setting changes) when those conditions are met. Automations are how you build a "DIY cockpit."

Autosens — Calculation of insulin sensitivity based on deviations over the past 8–24 hours. In FCL with autoISF, turn Autosens OFF — the two are incompatible.

Autotune — Suggests profile basal, carb ratio, and ISF adjustments based on historical data. Do not use with autoISF. Considered controversial.

B

basal rate — The baseline hourly insulin delivery defined in your profile, intended to hold bg stable in the absence of meals or disturbances.

bg — Blood glucose. (Technically the tissue glucose that CGMs measure, which lags blood glucose by a few minutes.)

bg_delta → see `delta`.

bg deviation — Difference between observed bg change and the change expected from insulin effects alone. Used by the loop to estimate carb absorption.

bg_ISF — autoISF component that strengthens ISF at high bg and softens it at low bg. Generally unused in FCL (set weights to 0.0). See §4.6.

bgAccel_ISF (acce_ISF) — autoISF component that strengthens ISF on detected glucose acceleration (a rising bg curve). The most important component in FCL — drives the first SMBs that replace your meal bolus. Tuned via `bgAccel_ISF_weight`. See §4.2.

bgBrake_ISF (brake) — autoISF component active during glucose deceleration (the rise slowing toward the peak). Usually half the weight of bgAccel_ISF. See §4.4.

bg source — The software source from which AAPS receives bg values. Typically a CGM app (Dexcom, xDrip+, BYODA, Juggluco for Libre 3).

Boost — AAPS-derived dev variant with its own meal-handling algorithm. Uses delta-based acceleration detection. See §8.3.

bolus calculator — Tool in AAPS (and equivalent in iOS Loop) that suggests a bolus size based on carbs, IC, ISF, and current iob. Not used in FCL.

BYODA — Build Your Own Dexcom App. Lets you build your own Dexcom reader app and pass smoothed bg values to AAPS or xDrip+, while still using Clarity®.

C

calculator → see `bolus calculator`.

CGM — Continuous Glucose Monitor. Sensor + transmitter + app that provides bg values to the loop.

cob — Carbs On Board. Remaining carbs available for absorption. Always zero in FCL (no carb entry).

connectivity — Bluetooth or WLAN connections between loop components (phone, pump, CGM, watch). Must be reliable 24/7 for FCL.

control of bg — The "boating-like" control problem of balancing carb absorption against insulin activity. Aggressive control risks hypos on turns; too-gentle control misses the target.

correction factor — iOS Loop term; analogous to the oref SMB delivery ratio. Scales the size of auto-boluses.

customization for iOS Loop — Advanced dev features available to iOS Loop users.

D

deceleration — The bg rise slowing as it approaches its peak. Triggers `bgBrake_ISF`.

delivery limit — Maximum bolus/basal rates permitted by your AAPS settings.

delivery ratio → see `SMB delivery ratio`.

delta (bg_delta) — Change in bg over the last 5 minutes. Visible in the AUTO ISF tab. - `d5` (or "delta") — last 5 minutes. - `short avg delta (d15)` — average over last 3 deltas (15 minutes). - `long avg delta (d45)` — average delta between 15 and 45 minutes back.

dev variant — Pre-release or side-branch version of AAPS, iAPS, or Trio. Includes experimental features not yet in the main (Master) release. autoISF itself is a dev variant.

Dexcom → see `G6`, `G7`, or `Dexcom ONE`.

DIA (Duration of Insulin Action) — How many hours your insulin remains active. Set in your profile. Critical for loop accuracy.

disturbance — Any factor that pushes bg away from a steady-state baseline — meals, exercise, fever, hormones, stress, and many others. Your profile handles the baseline; disturbances are managed by the loop.

DIY — Do It Yourself. Open-source, user-built looping ecosystem.

DIY cockpit — The constellation of buttons and Automations a user builds on the AAPS home screen for custom pre-programmed responses. See §5.4.

Dual Hormone Loop — Future system that adds a second hormone (glucagon or analogue) alongside insulin. Enables more aggressive insulin treatment without hypo risk. See §8.5.

dura_ISF — autoISF component active during persistent high-bg plateaus. Boosts ISF with both duration and height above target. Most relevant for fatty meals. Tuned via `dura_ISF_weight`. See §4.5.

dynamic carb absorption — The oref algorithm's continuous calculation of carb absorption based on observed bg deviations and insulin activity. Allows UAM/FCL operation without carb inputs.

dynamic carb ratio — Automatic IC adjustment based on bg level and recent TDD. Available in Trio/iAPS. Not used in oref FCL.

dynamicISF — Automatic ISF adjustment based on bg level and recent TDD. Do not use with autoISF. Covers up profile errors that FCL requires to be correctly set.

dynamic iobTH — autoISF's automatic modulation of iobTH based on active TT and exercise mode. See §3.4.

dynamic bg target — Loop feature that changes bg target based on detected sensitivity or resistance. Generally disabled in FCL to avoid interaction with the even/odd target logic.

E

EatingSoon TT — A low temporary target (e.g., 74 mg/dL) set before a meal so the loop delivers insulin sooner and the bg has a lower starting point. See §3.5.

EatingNow — AAPS-derived dev variant with a Meal Announcement workflow. See §8.3.

eCarbs — "Extended Carbs." Carb inputs with an absorption-time tail. Used in iOS Loop; not useful in oref loops.

Emulator — Python tool for replaying AAPS logfiles on a PC or phone. Enables retrospective analysis and what-if experimentation. Chapters 13–14.

even/odd target → see `odd target`.

exercise mode — Loop mode that limits iob and softens ISF during exercise. In autoISF, activated by yellow exercise button + elevated TT above profile target. Tunable via `half_basal_exercise_target`. See §6.2.

F

FCL (Full Closed Loop) — Looping mode where the user gives no bolus and enters no carbs. The loop handles everything automatically. Main subject of this book.

FPU (Fat-Protein Units) — Late carb-like insulin need from fat and protein content of a meal. Managed in FCL by dura_ISF (§4.5) or by temporary %profile switches.

G

G6 / G7 / Dexcom ONE — Dexcom CGM variants. G6 is the gold standard for FCL as of early 2026; production ends mid-2026.

glucagon — Counterregulatory hormone used in dual-hormone loops to prevent hypos. See §8.5.

GLP-1 / GIP receptor agonists — Drugs like Tirzepatide (Mounjaro®) that slow gastric emptying. Make FCL easier. See §8.6.

H

half-basal exercise target (HBET) — AAPS Preferences value that tunes the exercise-mode sensitivity ratio. Lower values produce more aggressive softening. See §6.2.

HbA1c — Long-term average blood glucose marker. Reported as %. Medical community commonly targets < 7.0%.

HCL (Hybrid Closed Loop) — The loop manages basal and between-meal corrections; the user bolusses for meals. Starting point for transitioning to FCL.

I

iAPS — iPhone-based oref loop (fork of the original iAPS). Has a dev port of autoISF. See Chapter 1 caveats.

IC (Insulin-to-Carb ratio) — Grams of carbs covered by 1 unit of insulin. Set in your profile. Critical for HCL; used by the loop in FCL for retrospective carb absorption calculations.

individualized tuning — The per-person calibration of all loop parameters based on observed data. The entire setup project in Chapters 3–8 is individualized tuning.

insulin activity — Insulin effect in the next 5 minutes, above profile basal supply (can be negative). Shown as the thin yellow line in AAPS.

insulin counteraction effect (ICE) — iOS Loop equivalent of bg deviation. Describes how observed bg changes differ from expected.

insulin kinetics — The rising-then-falling shape of insulin's activity curve. Varies by insulin type. Represented in AAPS by DIA and peak-time settings.

insulin required — The oref algorithm's calculation of how much additional insulin is needed, based on bg, iob, cob, and predictions. Distributed across SMBs and temp basal.

integral correction effect (IRC) — iOS Loop feature that adjusts predictions based on historical prediction errors.

iob — Insulin On Board. Units of insulin currently available to become active within the DIA window.

iob delta — Change in iob over recent minutes. Used by the loop to estimate what was actually absorbed.

iobTH — iob threshold. The iob ceiling above which SMBs are blocked. The most important safety setting in FCL. See §3.4.

iOS Loop — iPhone-based DIY loop using MPC (Model Predictive Control). Requires precise carb inputs — no UAM or FCL. Different algorithm from oref.

ISF (Insulin Sensitivity Factor) — Expected bg decrease from 1 unit of insulin. The single most important parameter in oref loops. Set in your profile; modulated dynamically by autoISF.

ISF_weight — Tunable factor in autoISF that scales each component's effect on profile_ISF. Five weights: `bgAccel_ISF_weight`, `pp_ISF_weight`, `bgBrake_ISF_weight`, `dura_ISF_weight`, and the `bg_ISF` weights.

J

Juggluco — Third-party app that passes Libre 3 raw values to AAPS in 1-minute mode.

L

LGS (Low Glucose Suspend) — AAPS safety feature that reduces basal when bg is dropping. Part of AAPS Objective 6.

Libre 2 / Libre 3 — Abbott's factory-calibrated CGMs. Alternatives to Dexcom. Libre 3 + Juggluco enables 1-minute mode in autoISF.

log files — Record of AAPS actions; used for troubleshooting, debugging, and Emulator replay.

logarithmic dynamicISF — Default dynamicISF variant; strengthens ISF logarithmically with rising bg.

Loop → see `iOS Loop`.

Loop Caregiver app — Remote monitoring/control app for iOS Loop (parent/child scenarios). Android equivalent: AAPS Client.

Loop Follow — Remote data viewer for iOS Loop.

M

MA → see `Meal Announcement`.

MAR (Minimum Absorption Rate) — iOS Loop carb-absorption floor. Not relevant to oref loops.

Master — The latest official release of AAPS (or of other loops). The stable, recommended version for most users. Not the same as dev variants.

maxIOB — Hard safety limit. AAPS Preferences setting for the maximum total iob the loop will ever allow.

MDI (Multiple Daily Injections) — Traditional insulin-pen therapy (no pump). Fallback if your looping system fails.

Meal Announcement (MA) — Closed-looping mode where you give a small pre-bolus at meals but don't count carbs. Middle ground between HCL and FCL. Chapter 7.

Meal Management — The art of balancing carb absorption against insulin activity for good post-meal bg control. Can be fully automated (FCL), partially announced (MA), or fully manual (HCL).

middleware — Custom-code add-ons for Trio and iAPS that supply the Automation-like functionality these platforms lack natively.

N

Nightscout / NSClient — Open-source web platform for remote bg monitoring, data logging, and remote control of AAPS.

Nightscout Reporter — Reporting tool that produces weekly/monthly summaries from Nightscout data. Useful for identifying outlier days and patterns.

normalTarget — Hard-coded reference point in AAPS (99 mg/dL from AAPS 4.2.1 onwards; was 100 before). Used in the exercise-mode sensitivity ratio formula.

O

occlusion — Partial or total cannula blockage that slows or stops insulin delivery. Produces hard-to-explain glucose rises and can destroy FCL performance.

odd target (even/odd target) — autoISF feature: odd-numbered bg targets (profile or TT) block SMBs entirely; even-numbered allow them. Your emergency SMB brake. See §2.4.

Open Source — Transparent, collaboratively-developed software available on GitHub. All DIY loops are open source.

open loop — Mode where the loop recommends actions but doesn't enact them — the user must approve each SMB/TBR. AAPS Objective 5.

oref — The algorithm family used in AAPS, Trio, and iAPS. Derived from the OpenAPS reference implementation. Distinct from iOS Loop's MPC algorithm.

override (iOS Loop) — Temporary %sensitivity change. Oref equivalent: profile switch.

P

parabola fit — Mathematical technique autoISF uses on recent CGM values to detect acceleration. More precise than raw delta thresholds.

patient type — AAPS setting that caps maxIOB based on user category (child, teen, adult). Designed to prevent dangerous overrides.

peak (bg peak) — Highest bg value reached after a meal. A major tuning target: lower peaks → easier late-meal management.

pod — Disposable insulin-delivery module for pod-based pumps (Omnipod, etc.).

pp_ISF — autoISF component active during the post-prandial (post-meal) steep linear rise. Second most important weight after bgAccel_ISF. See §4.3.

pre-bolus — Meal bolus given 5–20 minutes before eating, to give insulin a head-start against carbs. Used in HCL and MA mode; not in FCL.

predictions — Future bg projections calculated by the loop. In oref: eventualBG, IOBpredBG, ZTpredBG, COBpredBG, UAMpredBG (for different scenarios).

profile — Basic treatment settings (basal schedule, DIA, IC, ISF schedule, bg target). The foundation on which autoISF builds.

profile switch (%profile) — Temporary change of profile (e.g., 80% for exercise, 130% for illness). Multiplies basal, ISF, and iobTH proportionally.

R

readthedocs — Online documentation site for each DIY loop project.

recommended dose — iOS Loop equivalent of oref's insulin required.

remote control — Parent-operating-child or caregiver-operating-user setups using NSClient or Loop Caregiver.

resistance (insulin) — Above-normal insulin need. Common after fatty meals, during illness, with hormonal changes.

retrospective correction effect — iOS Loop feature that learns from recent prediction errors to improve future predictions.

roller-coaster — Glucose pattern with large swings up and down. Usually indicates over-aggressive ISF or unstable settings.

S

sens — The effective ISF after all modulations, displayed in the AUTO ISF tab.

sensitivity — Below-normal insulin need. Common after exercise or on days of high activity.

sensitivity adaptation — Methods for tracking and adjusting to sensitivity changes: Autosens, dynamicISF, Activity Monitor, autoISF, or manual temp %profile switches.

sensitivity detection — Calculation of sensitivity from bg deviations. Used by Autosens.

sensor noise — Unstable or jumpy CGM readings. Problematic for aggressive delivery settings and FCL acceleration detection.

sigmoid (dynamicISF) — Dev variant of dynamicISF with S-curve ISF response. Not recommended for Trio/iAPS beginners.

SMB (Super Micro Bolus) — Small boluses delivered by the loop, faster than temp basal. In HCL, limited to ~120 min of basal; in FCL, extended via `smb_max_range_extension`. iOS Loop equivalent: autoBolus.

SMB delivery ratio — Fraction (0–1) of calculated `insulin required` delivered per SMB cycle. AAPS default 0.5; raised to 0.6–0.7 in FCL. Values > 0.75 not recommended (CGM jitter sensitivity).

SMB range extension — Multiplier on the base 120-min-of-basal SMB size limit. Widened in FCL (typically 2.0–3.0) so SMBs can replace meal boluses. See §3.1.

smoothing — Algorithms that reduce CGM noise by averaging or interpolating values. Useful but costs you the earliest signs of rises. Configured in AAPS Configuration Builder.

source code — The underlying code of a DIY loop. Open source = freely readable, modifiable, and branchable on GitHub.

T

TAI — Trio + autoISF. The autoISF-integrated dev branch of Trio for iPhone.

TBR (Temp Basal Rate) — Temporary adjustment to basal, expressed as % of profile. Slower bg control than SMBs but always running.

TDD (Total Daily Dose) — Total insulin (bolus + basal) per day. Rough characterization of insulin need. Spikes during occlusions.

TIR (Time In Range) — Fraction of time bg is within a target range (typically 70–180 mg/dL). The primary performance metric for looping.

TITR (Time In Tight Range) — Time within 70–140 mg/dL. Used for more ambitious %TIR goals; usually requires HCL rather than FCL.

Tirzepatide / Mounjaro® → see `GLP-1 receptor agonist`.

Trio — iPhone-based oref loop. Has a dev port of autoISF.

Tsunami — AAPS-derived dev variant with a Meal Announcement workflow. See §8.3.

TT (Temporary Target) — Temporary bg target override. Used for exercise (high TT), EatingSoon (low TT), anti-hypo snack (odd TT for SMB shutoff), and many other cases.

tuning → see `individualized tuning`.

U

UAM (Un-Announced Meal) — oref's ability to detect rises without carb inputs and respond with SMBs. The basis of both FCL and the SMB+UAM algorithm in modern AAPS.

UTC / UTZ / CET — Time zones. AAPS logfiles use UTC internally; local-time display is a phone setting. For emulator work, subtract your offset from local time to get UTC.

V

vanilla — Informal term for unmodified Master AAPS (or other unmodified loop releases).

virtual pump — AAPS test option for running loop logic without a physical pump attached. Used in AAPS Objectives.

W

_weight → see `ISF_weight`.

wiki → see `readthedocs`.

X

Xcode — Apple's developer software needed to build and maintain a personal copy of iAPS or iOS Loop.

xDrip+ — Open-source CGM reader app that passes (optionally smoothed) values to AAPS. Also a rich alarms platform.

Y

YDMV (Your Diabetes May Vary) — Community reminder that responses to interventions vary person-to-person. Don't treat any single example as universal.

Z

zero-tempting — Temp basal at 0% (no basal insulin delivery). Often applied by the loop after large SMBs or when bg is heading toward target. Limits the loop's ability to respond quickly to new highs — one reason dual-hormone loops are attractive.

* * *

Appendix B — Case Studies Index

The case studies live alongside the source material on the author's GitHub. They're numbered by the section they illustrate. This index lists the case studies referenced throughout this book.

> 📁 *Where to find them: https://github.com/bernie4375/FCL-potential-autoISF-research — look for files named* `case_study_X.Y_*.pdf`.

Chapter 1 — Prerequisites

Case Study	*Title*	*What it shows*
1.1	Occlusion	25% TIR loss on a cannula-failure day; why early cannula changes matter
1.2	Comparing insulins for FCL	Modeling comparison of Lyumjev, Fiasp, Apidra, Humalog in FCL
1.3	Jumpy CGM	How CGM artefacts can be misinterpreted as meal starts
1.4	Lost pump connection	Why Bluetooth stability matters around meals
1.5	Permanent CGM values w/ 2× G6	Overlapping G6 sensors as the gold-standard CGM setup
1.6	When catastrophe strikes	Multi-failure scenarios and recovery
1.7	MDI when the loop dies	Fallback to pen-based therapy when the loop is unavailable
1.8	Libre 3 / 1-minute case	Placeholder — call for a case study from a Libre 3 user

Chapter 4 — Meals and ISF Weights

Case Study	Title	What it shows
4.1	Pizza	Worked example of tuning `_weights` for various autoISF factors
4.2	Low carb meals	Tuning for slow-absorption meals; role of dura_ISF
4.3	Hands-off FCL around Christmas	Real-world holiday week with full FCL

Chapter 5 — Modulating Loop Aggressiveness

Case Study	Title	What it shows
5.2	Sweet snacks / Glühwein with DIY cockpit	Building a snack-announcement button and related Automations
5.3	Compression low	Night-time CGM artefacts and how to prevent over-reaction

Chapter 6 — Exercise and Activity

Case Study	Title	What it shows
6.2	Biking day with hi-carb lunch; DIY cockpit	The 3-Automation pattern for meal-before-exercise (§6.5) in practice

Chapter 7 — Advanced HCL with Meal Announcement

Case Study	Title	What it shows
7.1	Meal Announcement (5 year old)	MA-based setup for a young child

Chapter 9 — Performance Monitoring

Case Study	Title	What it shows
8.2	Futility of tuning based on 1 extreme meal	Negative example — over-optimization for one meal hurts others

Chapter 10 — Troubleshooting

Case Study	Title	What it shows
9.1	Un-intended loss of loop aggressiveness	Diagnosing silent aggressiveness regressions

Chapter 8 — Other Avenues to FCL

Case Study	Title	What it shows
13.1	Comparison: 1 month FCL, Automation vs. autoISF	Side-by-side comparison of the two AAPS FCL approaches
13.2	FCL using dynamicISF	Call for submission — no interpretable data yet
13.3	Boost-based FCL for a child	Boost in practice for paediatric looping